

Software Requirements Specification (SRS)

Attendance Tracker

Version: 1.0

Project Type: Python Mini Project 5

Frontend: HTML/CSS

Backend: Python (Flask)

Storage: CSV Files

Document Control

Prepared by: Md. Anjir Jaman

Date: 05-09-2026

Version: 1.0 – Initial draft

1. Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Intended User

2. Overall Description

2.1 Product Perspective

2.2 Main Features

2.3 Operating Environment

3. Functional Requirements

4. Data Requirements

5. User Interface Requirements

5.1 Register Trainee Section

5.2 Mark Attendance Section

5.3 Attendance Report Section

6. Backend Requirements

7. Data Storage Requirements

8. Non-Functional Requirements

9. Error Handling

10. System Constraints

11. Assumptions

12. Acceptance Criteria

13. Project Schedule and Milestones

14. Requirement Summary

1. Introduction

1.1 Purpose

The Attendance Tracker is a web-based application that helps a trainer or coordinator register trainees, mark their daily attendance, and review attendance reports.

The application will provide a simple HTML-based dashboard through which users can register trainees, mark attendance for a selected date, and view or export attendance reports.

Trainee and attendance records will be stored in local CSV files so that the data remains available after the application is closed.

1.2 Scope

The system will provide the following main functions:

- Register a trainee with a unique Trainee ID and Name
- Select a date for marking attendance
- Mark each trainee as Present or Absent for the selected date
- Calculate attendance totals and percentage per trainee
- Display an attendance report for all trainees
- Export the attendance report as a CSV file
- Store trainee and attendance records in local CSV files
- Load existing records when the application starts
- Validate user input

The front-end will be developed using basic HTML/CSS, while Python (Flask) will handle the application logic and data processing.

1.3 Intended User

The system is intended for a trainer, teacher, or coordinator who needs to track attendance for a small group of trainees. The user does not need advanced technical knowledge to operate the application.

2. Overall Description

2.1 Product Perspective

The Attendance Tracker will operate as a locally-hosted Python web application, served using Flask and accessed through a web browser at localhost. The system will use local CSV files for data storage. No external database or internet connection is required.

Basic system flow:

User → HTML/CSS Front End → Python (Flask) Backend → CSV files (trainees.csv, attendance.csv)

Figure 1: System Architecture

The frontend will collect user input and display information. The Python backend will process requests, perform validation, manage trainee and attendance records, and handle file operations. The CSV files will provide persistent storage.

2.2 Main Features

- Register Trainee
- Mark Attendance
- Attendance Report
- Export Report

2.3 Operating Environment

The application will require:

- A computer
- Python 3 with Flask installed
- A modern web browser
- A Python development environment such as VS Code or PyCharm
- Local file storage

3. Functional Requirements

FR-01: Register Trainee

The system shall provide a form for registering a trainee with a Trainee ID and Trainee Name.

The system shall require both Trainee ID and Trainee Name before registering.

The system shall prevent duplicate Trainee IDs.

The system shall display a success or error message after a registration attempt.

FR-02: Select Date

The system shall allow the user to select a date for marking or viewing attendance.

The system shall default to today's date if none is selected.

FR-03: View Trainees for Selected Date

The system shall display all registered trainees along with their attendance status for the selected date.

If no attendance record exists for a trainee on the selected date, the system shall default their displayed status to Present.

FR-04: Mark Attendance

The system shall allow the user to set each trainee's status to Present or Absent for the selected date.

The system shall save the attendance status for all trainees on the selected date when requested.

If an attendance record already exists for a trainee on that date, the system shall overwrite the existing status rather than create a duplicate record.

FR-05: Calculate Attendance Report

The system shall calculate, for each trainee, the total number of days marked, days present, days absent, and attendance percentage.

The system shall round the attendance percentage to one decimal place.

FR-06: View Attendance Report

The system shall display the calculated attendance report for all trainees in a table.

FR-07: Export Report

The system shall allow the user to export the attendance report as a downloadable CSV file.

The exported file shall include Trainee ID, Trainee Name, Total Days, Present, Absent, and Attendance %.

FR-08: Save Records

The system shall store trainee records and attendance records in local CSV files.

The storage files shall be updated after registering a trainee or saving attendance.

FR-09: Load Records

The system shall load existing trainee and attendance records from the CSV files when the application starts.

If a storage file does not exist, the system shall create it and start with an empty data set.

FR-10: Input Validation

The system shall reject empty Trainee ID or Trainee Name fields during registration.

The system shall reject duplicate Trainee IDs.

The system shall accept only "Present" or "Absent" as a valid attendance status.

The system shall display clear error messages when invalid information is submitted.

4. Data Requirements

Each trainee record shall contain:

Field	Description	Required
Trainee ID	Unique trainee identifier	Yes
Name	Trainee's full name	Yes

Each attendance record shall contain:

Field	Description	Required
-------	-------------	----------

Trainee ID	Identifier of the trainee this record belongs to	Yes
Date	The date of the attendance record, in YYYY-MM-DD format	Yes
Status	Either "Present" or "Absent"	Yes

5. User Interface Requirements

The application shall use a basic HTML/CSS frontend, presented as a single dashboard page.

5.1 Register Trainee Section

The section shall contain input fields for Trainee ID and Trainee Name, and a Register Trainee button.

5.2 Mark Attendance Section

The section shall contain a date selector and a table listing every registered trainee with a Present/Absent control for the selected date.

The section shall provide a Save Attendance button.

5.3 Attendance Report Section

The section shall display a table with Trainee ID, Trainee Name, Total Days, Present, Absent, and Attendance % for every trainee.

The section shall provide an Export Report button.

Example:

Trainee ID	Name	Total Days	Present	Absent	Attendance %
T001	John Doe	10	9	1	90%

6. Backend Requirements

The Python (Flask) backend shall:

- Serve the HTML front end
- Receive and validate trainee registration requests
- Receive and validate attendance submissions for a selected date
- Calculate attendance totals and percentages from stored records
- Generate a downloadable CSV report on request
- Read from and write to the CSV storage files
- Return appropriate results and messages to the front end

7. Data Storage Requirements

The system shall use two local CSV files for storage: trainees.csv and attendance.csv.

Example trainees.csv:

```
id,name  
T001,John Doe
```

Example attendance.csv:

```
trainee_id,date,status  
T001,2026-09-05,Present
```

Because a CSV file has no built-in way to update a single row, the backend shall read the full attendance file into memory, replace the matching trainee/date row (or add a new one), and rewrite the entire file whenever attendance is saved.

The system shall support:

- Creating trainee records
- Creating and updating attendance records
- Reading records for report calculation
- Persistent storage across sessions

8. Non-Functional Requirements

NFR-01: Usability

The interface should be simple, clear, and easy to navigate.

NFR-02: Performance

Normal operations should complete within a reasonable time for the expected small number of trainees and attendance records. Rewriting the full attendance CSV on every save is acceptable at this scale but would not remain efficient for a large trainee population.

NFR-03: Reliability

The system should correctly save and retrieve trainee and attendance information during normal operation.

NFR-04: Maintainability

The Python code should be organized clearly, separating routing, validation, CSV access, and report calculation, so the application can be modified or extended easily.

NFR-05: Compatibility

The frontend should work with a modern web browser.

NFR-06: Data Persistence

Trainee and attendance information shall remain available after the application is closed and reopened.

9. Error Handling

The system shall handle common errors without crashing.

The system shall provide appropriate messages for:

- Missing Trainee ID or Trainee Name during registration
- Duplicate Trainee ID
- Missing date when marking attendance
- An invalid attendance status value
- A missing or corrupted CSV storage file
- Exporting a report when no trainees are registered

After an error, the user should be able to correct the input and try again without restarting the application.

10. System Constraints

- The backend will be developed using Python and Flask.
- The frontend will use basic HTML/CSS, with no JavaScript framework.
- Data will be stored locally using CSV files.
- No internet connection is required for normal operation.
- No external database is required.
- The system is intended for a small number of trainees and attendance records.
- The application will run locally through a web browser.
- The project will remain within the scope of a Python mini project.

11. Assumptions

- Python 3 and Flask are installed on the development computer.
- A modern web browser is available.
- The user has permission to read and write files in the project directory.
- Trainee IDs are chosen to be unique by the user.
- The CSV storage files are accessible to the application.

12. Acceptance Criteria

The project will be considered complete when:

- The HTML frontend loads successfully.
- A trainee can be registered with a unique Trainee ID.
- Duplicate Trainee IDs are prevented.
- Attendance can be marked for a selected date.
- Marking attendance again for the same date updates rather than duplicates the record.
- The attendance report displays correct totals and percentages.
- The report can be exported as a CSV file.
- Records are saved to and loaded correctly from the CSV files.
- Invalid input is handled correctly without crashing.
- Appropriate success and error messages are displayed.
- All major functions pass their test cases.

13. Project Schedule and Milestones

The project is scheduled to be completed in a single working session (approximately 6 hours), broken down by task as shown in Figure 2. The schedule follows the SDLC sequence defined in this document: requirements review, design, implementation, testing, and reporting.

Task	9am	10am	11am	12pm	1pm	2pm
Set up project, CSV load/save helpers						
Register Trainee form + route						
Mark Attendance UI + route						
Attendance calculation logic						
Report display + CSV export						
Testing + write report						

Figure 2: Development schedule

14. Requirement Summary

FR-01	Register trainee	FR-06	View report
FR-02	Select date	FR-07	Export report
FR-03	View trainees for date	FR-08	Save records
FR-04	Mark attendance	FR-09	Load records
FR-05	Calculate report	FR-10	Validate input
NFR-01	Usability	NFR-04	Maintainability
NFR-02	Performance	NFR-05	Compatibility
NFR-03	Reliability	NFR-06	Data persistence